Automated Reasoning over Scoped Syntaxes with Binding in Agda

Marius Weidner
University of Freiburg, Germany

Abstract

Mechanizing the metatheory of programming languages in proof assistants inevitably involves recurring challenges related to variable binding and substitution. Each formalization requires proving standard substitution lemmas, such as those characterizing the behavior of composition and the interaction of substitutions with binders.

Historically, substitution has been treated as an meta-level operation. To formalize this, explicit substitution calculi were introduced to bring substitutions down to a syntactic level. The σ-calculus provides syntactic reduction rules for terms and substitutions, ensuring confluence and strong normalization, thereby enabling equations between them to be solved by comparing their normal forms. By applying the reduction rules until a normal form is reached, we obtain an automatic procedure that eliminates the need for manually choosing the correct substitution lemmas.

To apply this automatic procedure within Agda proofs, the σ-calculus rules can be registered as native reduction rules using Agda's rewrite mechanism. Embedding the σ-calculus rules as reduction rules in Agda is challenging because it requires precise control over the reduction behavior of definitions, substitutions must be defined structurally in terms of renamings, and multiple variable sorts must be supported. We address the latter by adapting the σ-calculus to intrinsically scoped and multi-sorted syntax. We first treat renamings and substitutions separately, and then present a uniform treatment that allows results about both to be proven once, avoiding duplication. In both cases, thanks to the σ-calculus laws embedded as rewrite rules, equations are solved automatically by reduction, yielding definitional equalities in cases where substitution lemmas would otherwise need to be applied manually.

2012 ACM Subject Classification Author: Please fill in 1 or more \ccsdesc macro

Keywords and phrases Explicit Substitutions, Sigma Calculus, Kits, Agda

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Mechanizing the metatheory of programming languages in proof assistants inevitably involves recurring challenges related to variable binding and substitution. While such details are usually ignored in pen-and-paper proofs, their mechanization in proof assistants requires precise definitions and a significant amount of manual effort.

Historically, substitution has been treated as an meta-level operation. To formalize substitutions, explicit substitution calculi [1] were introduced to bring substitutions down to a syntactic level. These calculi define substitution behavior through a set of reduction rules. With the σ-calculus, a terminating, confluent [12] and complete [23] rewrite system for λ-calculus terms with substitution has been discovered. Repeatedly applying the rules left-to-right yields a normal form, which can then be compared for equality.

The σ-calculus laws can be generalized to custom syntaxes without much difficulty. Consequently, it is possible to use the σ-calculus laws in proof assistants to automatically generate proofs for equations in which only syntactic expressions and substitutions appear. The creators of Autosubst [24] were one of the first to realize this opportunity and provided a tactic that applies the σ-calculus rules to solve goals in Rocq (formerly Coq). Autosubst 2 [27] improves on its predecessor by addressing three fundamental limitations.

© Marius Weidner;
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

First, Autosubst originally only supported unscoped syntax. In Rocq, it is possible define intrinsically scoped syntax, represented as a set of terms indexed by their free variables. Variables are restricted to de Bruijn indices smaller than the index that represents the number of free variables. Hence, every syntax tree built in this style is well-scoped by construction. Autosubst 2 supports intrinsically scoped syntaxes directly.

Second, combining several syntactic categories that have variables can cause a combinatorial explosion of near-identical substitution definitions and lemmas. To address the combinatorial explosion problem, Autosubst 2 introduces vector substitutions, which bundle substitutions over different variable sorts into a single structure.

Third, in a proof assistant setting, it is necessary to distinguish between renamings, substitutions mapping variables to variables, and general substitutions, which map variables to terms, in order to obtain structurally recursive definitions [2]. The developers of Autosubst 2 recognized that maintaining the separation in full generality is useful, as there are situations where reasoning about renamings alone is required. To accommodate this, they extended Autosubst with first-class renamings, effectively duplicating the substitution infrastructure to handle renamings independently.

In Agda, there are several options for defining syntax: extrinsically scoped and typed syntax, intrinsically scoped but extrinsically typed syntax, and intrinsically scoped and typed syntax. We argue that the intrinsically scoped but extrinsically typed approach represents the sweet spot between the two extremes. In contrast to intrinsically typed syntax, scoped syntax allows all syntactic categories to be combined into a single multi-sorted and scoped data type, indexed not only by its free variables, but also by the syntactic categories. Using a multi-sorted and scoped syntax avoids the combinatorial explosion and the need for vector substitutions, as substitution can be defined uniformly across all sorts.

As we will see, adapting the σ-calculus with first-class renamings to a setting with multi-sorted, scoped syntax is relatively straightforward. In particular, when we are careful with the reduction behavior of our definitions, Agda's rewrite mechanism [11] allows the σ-calculus laws to be registered as native reduction rules, yielding definitional equalities where otherwise explicit application of substitution lemmas would be required.

Unfortunately, the need for renamings causes duplication. Lemmas, such as substitution-preserves-typing, must be proven twice, once for renamings and once for substitutions, with the latter relying on the former. To mitigate the duplication issue, we build on the notion of Kits [20], a structure that captures the behavior common to both variables and terms, originally developed in an intrinsically typed setting and later extended to multi-sorted, scoped syntax [22]. From Kits, we can define Maps, a uniform structure for renamings and substitutions. Our work adapts the σ-calculus laws to Kits and Maps over multi-sorted and scoped syntax, treating renamings and substitutions uniformly and shows how to embed the laws as reduction rules into Agda via rewrite rules. The resulting framework constitutes the first version of the Agdasubst library.

Structure

The paper is organized into several sections. Section 2 introduces the theoretical background and tools. In **??**, we discusses closely related work and outline our contributions more precisely. Section 3 presents an embedding of the σ-calculus as native reduction rules into Agda for multi-sorted, scoped syntax, treating renamings and substitutions separately. Section 4 explains the internals of the Agdasubst library, where renamings and substitutions can be treated uniformly. Section 5 illustrates the use of the library through case studies. Finally, we outline directions for future work in Section 6, discuss related work in Section 7 and

conclude the paper in Section 8.

Contributions

The main contributions of this work are

- an embedding of the σ-calculus with first-class renamings into Agda, in a scoped and multi-sorted setting, using rewrite rules, exemplified with System F (Section 3);
- a novel version of the σ-calculus, formulating the σ-calculus laws in a Kit- and Map-based setting, where we effectively lift the distinction between variables and terms, as well as between renamings and substitutions, from the syntactic to the type level. The new laws form the core of the AGDASUBST library (Section 4).

The supplement contains the complete Agda code underlying this paper.

2 Preliminaries

This section introduces the concepts and tools required for the developments in this work. We summarize

- Agda and language features relevant to our work: instance arguments, rewrite rules and opaque blocks;
- The σ-calculus;
- Multi-sorted and scoped syntaxes in Agda.

2.1 Agda and Relevant Language Features

Agda is a dependently typed programming language and proof assistant based on Martin-Löf type theory [19]. It enables users to write programs and proofs in a functional programming language, verifying correctness through type checking.

Proofs in Agda become substantially easier when the relevant terms are definitionally equal. To decide whether two terms are definitionally equal, Agda computes their weak head normal forms. A propositional equality proof $a \equiv b$ between two definitionally equal terms a and b can then be established using the refl constructor. If two terms are not definitionally equal, the user must resort to manual equational reasoning, typically by pattern matching and explicit application of lemmas. This is cumbersome, as Agda usually lacks tactics that automatically discharge proof obligations using existing lemmas. Increasing the number of definitional equalities between terms therefore greatly simplifies proofs and reduces manual effort.

In addition to its well-known features such as parameterized and indexed inductive datatypes, dependent (co-) pattern matching [10], implicit arguments and records, Agda supports three less commonly used but important features relevant to this work.

Instance Arguments

Instance arguments, written inside double braces $\{\{\ldots\}\}$, are resolved automatically via an instance search algorithm. When a function expects an instance argument, Agda attempts to infer and insert it from values marked with the instance keyword. This mechanism enables type-directed programming patterns similar to type classes in Haskell [30]. As an example, consider the following definition of a record for types with a default value:

```
record Default (A : Set) : Set where
  field default : A
```

We can bring the default field into scope using the open ... {{...}} syntax and instruct Agda to automatically resolve suitable record instances for occurrences of this field via instance search:

```
open Default {{...}}
```

At this point, instances for different types may be declared:

```
instance
  default-ℕ : Default ℕ
  default-ℕ .default = 0
```

```
instance
  default-String : Default String
  default-String .default = ""
```

The two instances allow implicit usage of the default values for natural numbers and strings via instance search, through the default field:

```
_ : default ≡ 0
_ = refl
```

```
_ : default ≡ ""
_ = refl
```

On the left side, Agda infers from the type ℕ of 0 that the instance default–ℕ applies for default, automatically inserting it to conclude that the equation holds definitionally. The same reasoning applies to the equality on the right side, involving the empty string.

Rewrite Rules

In this work, the most important feature of Agda is user-defined rewrite rules [11, 9], which extend Agda's computation rules by integrating propositional equalities as reduction rules directly into type checking. A function definition in Agda can be registered as a rewrite rule if it is of the form

$$\mathsf{eq} : (x_1 : A_1) \ldots (x_k : A_k) \to \mathsf{f}\, p_1 \ldots p_n \equiv v$$

and satisfies the following three conditions:

1. The head symbol f must be a postulate, a defined function, or a constructor applied to fully general type parameters (i.e. the type parameters must be distinct variables)
2. Each variable $x_1, \ldots, x_k$ must occur at least once in a pattern position among $p_1, \ldots, p_n$[1].
3. The left side $\mathsf{f}\, p_1 \ldots p_n$ must be neutral, i.e., it should not reduce any further using existing definitions.

As an example, consider the right identity law for addition:

```
+-idᵣ : ∀ n → n + 0 ≡ n
+-idᵣ zero    = refl
+-idᵣ (suc n) = cong suc (+-idᵣ n)
```

The lemma satisfies the three required conditions. It is a defined function symbol that does not reduce, as addition is defined via structural induction on the left argument, and all bound variables occur on the left side of the equation. We can now register it as a rewrite rule using the rewrite pragma:

[1] The Agda documentation includes a precise definition for pattern positions: Link to Agda Documentation

```
{-# REWRITE +-idʳ #-}
```

From this point on, Agda treats $n + 0$ as definitionally equal to n:

```
_ : ∀ {n} → n + 0 ≡ n
_ = refl
```

To maintain consistency, Agda requires that all rewrite rules satisfy the three conditions stated above, and that the combined set of rewrite rules is both terminating and confluent. Termination is currently not checked by Agda, it is only observable indirectly when type checking fails to terminate. Confluence can be checked using the `--local-confluence-check` and `--confluence-check` flags, but both checks are necessarily conservative.

Opaque Blocks

Agda supports opaque definitions [16], which treat selected parts of the code as non-unfolding symbols. Normally, Agda unfolds all definitions automatically to produce as many definitional equalities as possible. While this behavior is desirable in most cases, we will require certain definitions to remain blocked, so that rewrite rules apply predictably to these symbols. In later opaque blocks, the type checker can be instructed to unfold previously blocked definitions, ensuring that their behavior remains accessible when needed.

A block is marked as opaque by preceding it with the opaque keyword. All definitions within the block become opaque, while their names and types remain visible and usable throughout the code:

```
opaque
  forty-two : ℕ
  forty-two = 42
```

In this case, forty-two is defined as the numeral 42, but outside the block it is treated abstractly. This means that although forty-two still has type ℕ and can be used in expressions and proofs, it does not reduce to 42 during definitional equality checks:

```
_ : forty-two ≡ 42
_ = {!   !} -- not provable.
```

Using unfolding reduces the definition to 42 again, as expected:

```
opaque
  unfolding forty-two

  _ : forty-two ≡ 42
  _ = refl
```

2.2 The Sigma Calculus

The σ-calculus [1] formalizes the behavior of substitutions through explicit, syntactic reduction rules. Of particular interest is the σ_{SP}-calculus [23], an extension of the σ-calculus that ensures confluence and forms the theoretical foundation of Autosubst. The σ_{SP}-calculus is sound, confluent [12], and complete [23], meaning that any equation involving terms and substitutions can be correctly proven or disproven by repeated application of its reduction

Symbols

Variables x

innermost variable	zero
outer variables	$\mathsf{suc}\ x$

Terms t

variable usage	$`x$
abstraction	$\lambda.\, t$
application	$t_1\ t_2$
substitution lookup	$x \mathbin{\&} \sigma$
substitution application	$t \cdots \sigma$

Substitutions σ

identity	id
extension	$t \cdot \sigma$
composition	$\sigma_1\ ;\ \sigma_2$
weakening	wk
lifting	$\uparrow\sigma \doteq ((`\mathsf{zero}) \cdot (\sigma\ ;\ \mathsf{wk}))$

Reduction

Definitional Laws

id	$x \mathbin{\&} \mathsf{id}$	$= `x$
$\mathsf{ext_Z}$	$\mathsf{zero} \mathbin{\&} (t \cdot \sigma)$	$= t$
$\mathsf{ext_S}$	$\mathsf{suc}\ x \mathbin{\&} (t \cdot \sigma)$	$= x \mathbin{\&} \sigma$
comp	$x \mathbin{\&} (\sigma_1\ ;\ \sigma_2)$	$= (x \mathbin{\&} \sigma_1) \cdots \sigma_2$
wk	$x \mathbin{\&} \mathsf{wk}$	$= `(\mathsf{suc}\ x)$

Traversal Laws

var	$(`x) \cdots \sigma$	$= x \mathbin{\&} \sigma$
lam	$(\lambda.\, t) \cdots \sigma$	$= \lambda.(t \cdots (\uparrow\sigma))$
app	$(t_1\ t_2) \cdots \sigma$	$= (t_1 \cdots \sigma)\ (t_2 \cdots \sigma)$

Interaction Laws

$\mathsf{comp\text{-}id_L}$	$\mathsf{id}\ ;\ \sigma$	$= \sigma$
$\mathsf{comp\text{-}id_R}$	$\sigma\ ;\ \mathsf{id}$	$= \sigma$
$\mathsf{associativity}$	$(\sigma_1\ ;\ \sigma_2)\ ;\ \sigma_3$	$= \sigma_1\ ;\ (\sigma_2\ ;\ \sigma_3)$
$\mathsf{distributivity}$	$(t \cdot \sigma_1)\ ;\ \sigma_2$	$= (t \cdots \sigma_2) \cdot (\sigma_1\ ;\ \sigma_2)$
$\mathsf{interact}$	$\mathsf{wk}\ ;\ (t \cdot \sigma)$	$= \sigma$
$\eta\text{-}\mathsf{id}$	$(`\mathsf{zero}) \cdot \mathsf{wk}$	$= \mathsf{id}$
$\eta\text{-}\mathsf{law}$	$(\mathsf{zero} \mathbin{\&} \sigma) \cdot (\mathsf{wk}\ ;\ \sigma)$	$= \sigma$

Monad Laws

$\mathsf{right\text{-}id}$	$t \cdots \mathsf{id}$	$= t$
$\mathsf{compositionality}$	$(t \cdots \sigma_1) \cdots \sigma_2$	$= t \cdots (\sigma_1 ; \sigma_2)$

Figure 1 The three-sorted σ-calculus

rules. We present our own three-sorted variant of the σ_{SP}-calculus, instantiated with the λ-calculus as the object language in Figure 1.

Our system closely follows the σ_{SP}-calculus. The syntax is sorted into de Bruijn variables, terms, and parallel substitution primitives. We use de Bruijn variables to obtain α-equivalence for free. Terms include the standard λ-calculus constructs, extended with the lookup in a substitution and application of a substitution to terms.

The Substitution primitives allow the construction of parallel substitutions, which are maps from variables to terms, and support the composition of multiple such substitutions. Unlike most presentations in the literature, we include explicit syntax for variable usage within terms. Because of the syntax for variable usage within terms, we require a distinction between suc and wk to shift variables by one index, depending on whether they appear in the sort of variables or substitutions. Applying a weakening substitution to a term, shifts all variables inside the term by one index. We also include a shorthand for lifting, which enables a substitution to be pushed under a binder. A lifted substitution replaces the bound variable with itself, leaving all occurrences of the variable inside the body of the binding construct unchanged, while weakening all terms in the substitution so that they correctly refer to the variables outside the binding construct.

Inspired by [26], we group the reduction rules into four categories.

- The **Definitional Laws** describe how variables are looked up in substitutions;
- **Interaction Laws** express equalities between substitution expressions;
- **Traversal Laws** describe how substitutions propagate through term syntax and under binders;
- **Monad Laws** describe the behavior of substitutions when applied to terms.

We deliberately do not include reduction rules for the λ-calculus itself, such as β-reduction, as done for example in the $\lambda\sigma$-calculus [12]. Our focus is not on the reduction of the object

```
data Kind : Set where
  ⋆ : Kind
data Type (n : ℕ) : Set where
  ‘_       : Fin n → Type n
  ∀[α:_]_  : Type (suc n) → Type n
  _⇒_      : Type n → Type n → Type n
data Expr (n m : ℕ) : Set where
  ‘_    : Fin m → Expr n m
  λx_   : Expr n (suc m) → Expr n m
  Λα_   : Expr (suc n) m → Expr n m
  _·_   : Expr n m → Expr n m → Expr n m
  _•_   : Expr n m → Type n → Expr n m
```

```
data Sort : Set where
  expr type kind : Sort
Scope = List Sort
Scoped = Scope → Sort → Set
data _∋_ : Scoped where
  zero : ∀ {S s} → (s :: S) ∋ s
  suc  : ∀ {S s s′} → S ∋ s → (s′ :: S) ∋ s
data _⊢_ : Scoped where
  ‘_      : S ∋ s → S ⊢ s
  λx_     : (expr :: S) ⊢ expr → S ⊢ expr
  Λα_     : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_]_ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  _·_     : S ⊢ expr → S ⊢ expr → S ⊢ expr
  _•_     : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_     : S ⊢ type → S ⊢ type → S ⊢ type
  ⋆       : S ⊢ kind
```

Figure 2 Classically Scoped Syntax vs. Multi-Sorted and Scoped Syntax

language, but on the behavior of substitutions applied to terms. The reduction strategy of the object language is part of its definition and is left unspecified. The only syntax-dependent rules we require are those traversing terms with substitutions.

We believe the three-sorted σ-calculus is terminating and confluent. Although we do not give a formal proof, it is a reasonable variant of the σ_{SP}-calculus. Renamings, needed to define substitutions structurally in Agda, break completeness of the system, when available as first-class syntactic constructs [27]. For this reason, we do not consider completeness a central property in this work. Still, we try to keep the space of solvable equations as large as possible.

2.3 Multi-Sorted and Scoped Syntaxes in Agda

Unlike the λ-calculus, many programming languages feature multiple syntactic categories. For instance, System F [14, 21] has expressions, types and kinds, with two of the three categories supporting variables. To avoid scoping-related errors, we aim for syntaxes that enforce correct scoping intrinsically. An intrinsically scoped syntax definition only permits the construction of object language terms that are correctly scoped by construction. In Agda, intrinsically scoped syntax is typically achieved by a syntax definition as a family of types indexed by the set of free variables.

On the left side of Figure 2, we present a standard intrinsically scoped syntax for System F. Types are indexed by the number of free type variables, and expressions are indexed by both the number of free type and expression variables. Variables are represented using de Bruijn indices of type Fin n, that is, natural numbers strictly less than the number of free variables. This representation ensures that only well-scoped terms can be constructed. Technically, the Kind type could be omitted, as all types, in the case of simple System F, are well-formed by construction.

This approach is simple but has drawbacks. Types and expressions are separate datatypes, so we would need to define multiple substitution operations: type-in-type, expr-in-expr, and type-in-expr. To avoid the combinatorial explosion of substitution operations, we define a uniform, multi-sorted and scoped syntax on the right side of Figure 2. Terms form a single

datatype indexed by a list of sorts, called Scope and ranged over by S, together with a target sort. A type $S \vdash s$ describes a term of sort s with free variables whose sorts are listed in S. Variable positions are represented by $S \ni s$, a proof that a variable of sort s appears in S. The proof acts as the de Bruijn index. Using the unified syntax, we can later define scoped and multi-sorted renamings and substitutions that operate uniformly across all sorts.

Due to the similarity in terminology, the above approach might be confused with Pure Type Systems [7, 6, 28], which describe a class of typed lambda calculi parameterized over a set of sorts, dependencies between these sorts, and rules for permitted quantifications. The key difference is that, in multi-sorted syntax, sorts differentiate syntactic categories, keeping terms of different sorts distinct. In contrast, in Pure Type Systems, sorts act as universe types, such as Set in Agda, and terms of different sorts belong to a single syntactic category.

3 The Sigma Calculus with Separate Renamings and Substitutions

This section presents a direct embedding of the σ-calculus rules with first-class renamings, following the conceptual theory underlying Autosubst 2, as native reduction rules in Agda using its rewrite mechanism. We use the multi-sorted and scoped version of SystemF from Figure 2 as an example for any custom syntax. The approach described in this section directly translates to any multi-sorted and scoped syntax definition.

3.1 Scoped and Multi-Sorted Renamings and Substitutions

We define renamings ($_\rightarrow^R_$) and substitutions ($_\rightarrow^S_$) uniformly over all variable sorts by indexing them over scopes, as shown in Figure 3. A renaming from scope S_1 to S_2 maps each variable of a given sort in S_1 to a variable of the same sort in S_2. Similarly, substitutions map variables to terms of the corresponding sort.

We first define the renaming type and its primitive operations: identity (id^R), weakening (wk^R) and extension ($_\cdot^R_$). Substitution primitives mirror those for renamings, but use the superscript S instead of R, except for weakening, which is strictly a renaming. The order of definitions is chosen carefully to obtain a structurally recursive definition. We use the structurally smaller weakening renaming to define lifting of substitutions ($_\uparrow^S_$).

To support all composition combinations of renamings and substitutions, we provide four forward composition primitives: renaming composed with renaming ($_;^{RR}_$), renaming with substitution ($_;^{RS}_$)[2], substitution with renaming ($_;^{SR}_$), and substitution with substitution ($_;^{SS}_$).

The traversal functions ($_\cdots^R_$ for renaming and $_\cdots^S_$ for substitution) are defined by structural induction on the syntax. Each clause applies a renaming or substitution to a object language term, lifting the renaming or substitution, when going under a binder. The clauses in both definitions correspond directly to the traversal laws of the σ-calculus, and therefore, we want these functions to reduce during type checking. When adapting this approach to arbitrary custom syntaxes, the traversal functions are the only definitions that must be provided by the user. All other definitions can be stated independently of the concrete syntax.

Renaming and substitution primitives are declared opaque, which blocks their reduction. This abstraction is crucial for rewriting the σ-calculus laws, where the primitives must act

[2] The first two composition operators can technically be merged into a single definition. Both are simply forward function composition, but we choose to keep them separated.

as uninterpreted symbols rather than as concrete functions. Blocking reduction in this way allows rewrite rules to apply predictably. The types for renamings and substitutions are declared opaque to prevent their misuse as actual functions. Renamings and substitutions should not behave as functions, because rewriting cannot be performed on a stuck function applied to an argument. Instead, we introduce explicit syntax for function application ($_\&^R_$ and $_\&^S_$) to allow rewriting on a renaming or substitution applied to a variable.

3.2 The Sigma Calculus with First-Class Renamings

While we could opt to restrict the relevance of renamings to the structural definition of substitutions and state the laws only for substitutions, it is often useful to reason about renamings directly, for instance in the context morphism lemma [15]. We therefore, similar to Autosubst, treat renamings as first-class constructs. Figure 4 presents the σ-calculus laws with first-class renamings. They all satisfy the requirements from Section 2.1 and qualify as rewrite rules.

The **Definitional Laws** now involve primitives for both renamings and substitutions. The **Interaction Laws** likewise cover all operators. Many of these laws are similar in shape. For instance, we require eight associativity laws. The **Traversal Laws** follow directly from the definition of traversal given in the previous section.

Most of the laws can be proven independently of the concrete syntax. In contrast, the **Monad Laws** must be shown by induction over the syntax and vary for each object language definition. We highlight syntax-dependent laws in red throughout this work.

[27] noticed that a further category emerges from the interaction between renamings and substitutions. The **Coincidence Laws** govern the case where substitutions are intertwined with renamings, but the renamings can be separated from them, and applied to a term separately. The coincidence and the coincidence-fold laws enable us to recover renamings from their embedding as substitutions via the identity substitution. Extracting arbitrary renamings from deeply nested substitutions would require a dedicated solving strategy, which we leave for future work but consider important for a more complete account.

In her PhD thesis, [26] proves the coincidence law by induction over the syntax. The law is not syntax-dependent when composition between renamings and substitutions is included in the theory and the monad laws are available:

$$
\begin{array}{ll}
t \cdots^S (\rho \,;^{RS}\, \mathsf{id}^S) & \equiv\langle\ \mathsf{sym}\ (\mathsf{compositionality}^{RS}\ t)\ \rangle \\
(t \cdots^R \rho) \cdots^S \mathsf{id}^S & \equiv\langle\ \mathsf{right\text{-}id}^S\ (t \cdots^R \rho)\ \rangle \\
t \cdots^R \rho & \square
\end{array}
$$

The remaining coincidence laws follow analogously.

The exact influence of coincidence laws on the metatheory of the rule set as a whole still needs to be examined in detail but it is believed their addition preserves termination and confluence [27]. Agda cannot determine that the set of rewrite rules is confluent using its built-in checks. To make the rules amenable to such analysis, a critical pair analysis in the spirit of the Knuth-Bendix completion algorithm [18] would be required to split the rules into components that Agda can recognize as collectively confluent.

To prove the σ-calculus laws presented in Figure 4, we need to unfold the definitions of the renamings and substitutions primitives. Laws colored blue can be proven syntax-independently, while laws colored in red need to be proven by structural induction over the syntax. Our implementation, found in the supplement, makes use of function extensionality, which is safe to postulate in Agda. It is known that with a little more effort, this assumption is not necessary.

3.3 Discussion

The laws presented in Section 3.2 are an extension of the classic σ-calculus rules to a setting with renamings. They fulfill the requirements to be registered as rewrite rules without further modification. This is a consequence of marking the relevant symbols opaque. Inside the proofs of the laws themselves, the definitions can still unfold when needed, while externally they behave as uninterpreted symbols and are reduced only by the σ-calculus laws.

The main drawback of the approach with separated renamings and substitutions is the unavoidable duplication of similar definitions and proofs. Associativity laws pose little difficulty as they can be stated independently of the syntax, but the definition of traversal and monad laws must be provided explicitly. These include two traversal definitions, two right-identity laws, and four compositionality laws, all similar in shape. Even if syntax-dependent definitions and laws are derived automatically, every subsequent result about substitutions would still need to be proven twice, once for renamings and once for general substitutions.

4 The Agdasubst Library

This section introduces the Agdasubst library. It builds on the ideas from Section 3 but removes the strict separation between variables and terms by the introduction of a construct that can represent either, which we call Kits. Using Kits, renamings and substitutions can be defined uniformly through a single notion, called Maps. With Kits and Maps in place, definitions and proofs from Section 3 that were previously similar in shape can now be unified.

We first recall the relevant definitions from the work of [22] in Sections 4.1 and 4.2. Most implementation details of Kits and Maps are omitted, except for those directly relevant to the σ-calculus laws. For a complete account of Kits, Maps, and their extensions, we refer the reader to [22]. We then show how the σ-calculus laws can be formulated in the setting of Kits and Maps and discuss the challenges in doing so. In particular, it is challenging to formulate the laws, such that they satisfy the requirements to be registered as rewrite rules.

The library, and the following chapter, abstracts over a set of sorts $Sort$: Set, a scoped and multi-sorted syntax _⊢_: List $Sort \to Sort \to$ Set, that must have a variable constructor ‘_: $S \ni s \to S \vdash s$.

4.1 Kits

A Kit is a structure that abstracts over terms and variables. The intended usage is to instantiate the Kit structure exactly twice, first for variables and then for terms. Definitions can be written uniformly, parameterized over a Kit, and are consequently applicable to both cases.

```
record Kit (k : Mode) : Set₁ where
  _∋/⊢ᴷ_ = image k
  field
    K-id/‘ : S ∋ s → S ∋/⊢ᴷ s
    K-‘/id : S ∋/⊢ᴷ s → S ⊢ s
    K-wk   : ∀ s′ → S ∋/⊢ᴷ s → (s′ ∷ S) ∋/⊢ᴷ s
    -- axioms ...

  open Kit {{...}}
```

```
data Mode : Set where
  Vᴹ Tᴹ : Mode

image : Mode → (List Sort → Sort → Set)
image Vᴹ = _∋_
image Tᴹ = _⊢_
```

To distinguish the two instances of Kits on the type level, Kits are indexed over a boolean-like structure Mode. The image function computes, given a fixed Mode, whether the Kit corresponds to a variable or term. The Kit record has multiple fields.

- K-id/‘ converts a variable into a variable or term $S \ni\!/\!\vdash^K s$. For the Variable Kit, $_\ni\!/\!\vdash^K_$ is instantiated to $_\ni_$, so the operation needs to act as the identity. For the Term Kit, it is instantiated to $_\vdash_$, so the operation corresponds to the variable constructor.
- K-‘/id converts an element of type $S \ni\!/\!\vdash^K s$ into a term of type $S \vdash s$, and works analogous to K-id/‘.
- K-wk shifts the de Bruijn indices in a variable or term. The newly introduced, unused variable zero can assume any sort s'. For variables, K-wk corresponds to the successor constructor suc. For terms, K-wk needs to apply a weakening renaming to the term.

The Kit record further includes fields for axioms that constrain its instantiation. We will not cover these axioms, because for both variables and reasonable syntaxes, all of them hold almost by definition.

Because rewrite rules cannot mention record fields directly, we provide for each field, except K-wk, a corresponding version without the K-prefix. These are declared as opaque functions, so they can be used as symbols in rewrite rules, while still be selectively unfolded to the actual fields when needed. Going forwards, we assume all opaque definitions unfolding any time throughout this section. It is only outside the library, when we have the σ-calculus laws in place, when we want definitions to act as uninterpreted symbols.

We can now define the usual operations for renamings and substitutions. The definitions are included directly inside the Kit record, so they are implicitly parameterized over a Kit K with Mode M and can be uniformly stated. All the primitives are marked opaque.

```
_→ᴷ_ : Scope → Scope → Set
S₁ →ᴷ S₂ = ∀ s → S₁ ∋ s → S₂ ∋/⊢ᴷ s

_&_ : ∀ {s} → S₁ ∋ s → S₁ →ᴷ S₂ → S₂ ∋/⊢ᴷ s
x & φ = φ _ x

id : S →ᴷ S
id s x = K-id/‘ x
```

```
_·_ : S₂ ∋/⊢ᴷ s → S₁ →ᴷ S₂ → (s ∷ S₁) →ᴷ S₂
(x/t · φ) _ zero    = x/t
(x/t · φ) _ (suc x) = φ _ x

_;wk : S₁ →ᴷ S₂ → S₁ →ᴷ (s ∷ S₂)
_;wk φ _ x = K-wk _ (φ _ x)

wk : S →ᴷ (s ∷ S)
wk = id ;wk
```

The type for Maps $S_1 \to^K S_2$ unifies renamings $S_1 \to^R S_2$ and substitutions $S_1 \to^S S_2$. For M specialized to V^M the image function computes to the type of variables and as a result the Map acts as renaming. Similarly, in the case that M is equal to T^M, the Map corresponds

to a substitution. The identity Map, as well as lookup and extension operations behave analogously in both the renaming and substitution cases.

Term traversal is not yet available, because it requires Kits to be defined first, as traversal is formulated for Maps directly, parameterized by a Kit. At the same time, traversal is needed to define composition for Maps, because composing with a substitution on the right relies on traversal, and composition is in turn required to define lifting:

```
_↑_ : S₁ →ᴷ S₂ → ∀ s → (s :: S₁) →ᴷ (s :: S₂)
φ ↑ s = (id/` zero) · (φ ;wk)
```

To break this dependency cycle, we introduce a generalized weakening operator _;wk, which can later be rewritten as an actual composition with the weakening primitive wk, which we also define in terms of the generalized version.

Similarly, once term traversal is available, the id/` and `/id operators can be expressed entirely in terms of the primitive operations. We also provide syntax to refer to Kits explicitly inside square brackets:

```
_∋/⊢[_]_ : Scope → Kit M → Sort → Set
S ∋/⊢[ K ] s = Kit._∋/⊢ᴷ_ K S s

_–[_]→_ : Scope → Kit M → Scope → Set
S₁ –[ K ]→ S₂ = Kit._→ᴷ_ K S₁ S₂

id[_] : (K : Kit M) → S –[ K ]→ S
id[ K ] = Kit.id K
```

To define the Term Kit instance, we need to know how a Map is applied to a term. The user has to provide the Kit-based traversal function and a proof that applying the identity Map leaves a term unchanged by structural induction over their syntax:

```
_⋯_ : ∀ {{K : Kit M}} → S₁ ⊢ s → S₁ –[ K ]→ S₂ → S₂ ⊢ s
⋯-right-id : ∀ {{K : Kit M}} → (t : S ⊢ s) → t ⋯ id[ K ] ≡ t
```

Equipped with the traversal function we can define the two intended instances for Kits:

```
instance
  V : Kit Vᴹ
  V = record
    { K-id/` = λ x → x
    ; K-`/id = `_
    ; K-wk   = λ s′ x → suc x
    -- axioms ...
```

```
instance
  T : Kit Tᴹ
  T = record
    { K-id/` = `_
    ; K-`/id = λ t → t
    ; K-wk   = λ s′ t → t ⋯ wk
    -- axioms ...
```

In the K-wk field for the Term Kit, we use the wk primitive from the Variable Kit. The Variable Kit is automatically resolved using instance search.

4.2 Composition

We aim to abstract over the four composition options for renamings and substitutions, enabling unified composition lemmas, such as one associativity law, instead of separate laws for each of the eight valid combinations. To this end, we define the ComposeKit record, parameterized over three Kits. Inside the record, we can later define a single, more general composition of a K_1-Map with a K_2-Map, yielding a $K_1 \sqcup K_2$-Map.

```
record ComposeKit (K₁ : Kit M₁) (K₂ : Kit M₂) (K₁⊔K₂ : Kit M₃) : Set₁ where
  private instance _ = K₁; _ = K₂; _ = K₁⊔K₂
  field
    C-_&/⋯_ : S₁ ∋/⊢[ K₁ ] s → S₁ –[ K₂ ]→ S₂ → S₂ ∋/⊢[ K₁⊔K₂ ] s
    -- axioms ...
```

The operation C-_&/⋯_ unifies lookup and traversal. It takes a variable or term x/t (according to K_1) and a renaming or substitution φ (according to K_2), and applies φ to x/t, yielding a variable or term (according to $K_1 \sqcup K_2$). From unified lookup/traversal, for which we also provide an opaque function version without the C-prefix, we derive Map composition:

```
_;_ : S₁ –[ K₁ ]→ S₂ → S₂ –[ K₂ ]→ S₃ → S₁ –[ K₁⊔K₂ ]→ S₃
(φ₁ ; φ₂) _ x = (x & φ₁) &/⋯ φ₂
```

The Map composition operator, such as the unified lookup/traversal operator, is implicitly parameterized over a Compose Kit. It takes an renaming or substitution φ_1 (according to K_1) and a renaming or substitution φ_2 (according to K_2) as arguments, and composes it to a renaming or substitution (according to $K_1 \sqcup K_2$). When x & φ_1 yields a variable, we need to look up the resulting variable in φ_2, or if we have a term, we traverse the term with φ_2. We also define versions of both the unified lookup/traversal operator and the composition operator where the Compose Kit is given explicitly: _;[_]_ φ_1 C φ_2 = ComposeKit._;_ C φ_1 φ_2 and _&/⋯[_]_ x/t C φ_2 = ComposeKit._&/⋯_ C x/t φ_2.

As with Kits, we now require a user-provided proof about traversal. We need the user to show that the chosen syntax and traversal function satisfy compositionality. This proof can only be established by induction on the syntax:

```
⋯-compositionality : ∀ {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{C : ComposeKit K₁ K₂ K₃}}
  (t : S₁ ⊢ s) (φ₁ : S₁ –[ K₁ ]→ S₂) (φ₂ : S₂ –[ K₂ ]→ S₃) →
  (t ⋯ φ₁) ⋯ φ₂ ≡ t ⋯ (φ₁ ; φ₂)
```

Finally, to cover the four possible instances of Compose Kits, we provide two generic instances that suffice:

```
instance                                  instance
  C^R : {{K : Kit M}} → ComposeKit V K K  C^S : {{K : Kit M}} {{C : ComposeKit K V K}} → ComposeKit T K T
  C^R = record { C-_&/⋯_ = _&_ -- axioms  C^S = record { C-_&/⋯_ = _⋯_ -- axioms ...
```

The C^R instance corresponds to the Compose Kit for composition between a renaming and another K-Map. Similarly, C^S covers the composition between a substitution and another K-Map, and assumes that composition of a K-Maps with renamings is already available.

The third parameter $K_1 \sqcup K_2$ of a Compose Kit can be understood as a functional dependency, uniquely determined by the choice of K_1 and K_2. It is instantiated to V if and only if both K_1 and K_2 are V, otherwise it is T. In other words, only renamings compose to a renaming, while every other case yields a substitution. We can put this more algebraically: if φ_1 is a K_1-Map and φ_2 is a K_2-Map, then the result is a $(K_1 \sqcup K_2)$-Map, where $\sqcup$ refers to the join-semilattice on $\{V, T\}$ generated by $V < T$. A natural question is why we do not simply compute $K_1 \sqcup K_2$. Computing $K_1 \sqcup K_2$ is in fact possible and even necessary to define the σ-calculus rules so that they still adhere to restrictions on the shape of rewrite rules.

4.3 Structural Kit Order

We define a least upper bound operation on Modes that mirrors the behavior of composing two Maps, indexed by Kits of these Modes, inside an opaque block:

```
_⨆_ : Mode → Mode → Mode
V^M ⨆ V^M = V^M
V^M ⨆ T^M = T^M
T^M ⨆ V^M = T^M
T^M ⨆ T^M = T^M
```

By case analysis, all the join-semilattice laws are easily verified:

```
⨆-idem      : M ⨆ M              ≡ M
⨆-assoc     : (M₁ ⨆ M₂) ⨆ M₃ ≡ M₁ ⨆ (M₂ ⨆ M₃)
⨆-comm      : (M₁ ⨆ M₂)          ≡ (M₂ ⨆ M₁)
⨆-bot-right : M ⨆ V^M ≡ M
⨆-bot-left  : V^M ⨆ M ≡ M
⨆-top-right : M ⨆ T^M ≡ T^M
⨆-top-left  : T^M ⨆ M ≡ T^M
```

All laws can be embedded as rewrite rules in Agda, except for ⨆-comm, which is evidently non-terminating. The lattice laws for Modes must be registered as rewrite rules to define the semilattice laws for Kits. Without the rewrite rules, explicit type coercions would be necessary, because Modes appear in the type of Kits. We then define the least upper bound for Kits:

```
_⊔_ : (K₁ : Kit M₁) (K₂ : Kit M₂) → Kit (M₁ ⨆ M₂)
_⊔_ {V^M} {V^M} K₁ K₂ = V
_⊔_ {V^M} {T^M} K₁ K₂ = T
_⊔_ {T^M} {V^M} K₁ K₂ = T
_⊔_ {T^M} {T^M} K₁ K₂ = T
```

and prove the join-semilattice laws:

```
⊔-idem      : {{K : Kit M}} → K ⊔ K ≡ K
⊔-assoc     : {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} →
  (K₁ ⊔ K₂) ⊔ K₃ ≡ K₁ ⊔ (K₂ ⊔ K₃)
⊔-comm      : {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} →
  (K₁ ⊔ K₂) ≡ subst Kit (⨆-comm {M₂} {M₁}) (K₂ ⊔ K₁)
⊔-bot-right : {{K : Kit M}} → K ⊔ V ≡ K
⊔-bot-left  : {{K : Kit M}} → V ⊔ K ≡ K
⊔-top-right : {{K : Kit M}} → K ⊔ T ≡ T
⊔-top-left  : {{K : Kit M}} → T ⊔ K ≡ T
```

The ⊔-comm law requires the transfer lemma in its type, because ⨆-comm for Modes could not be rewritten earlier. All other laws can be registered as rewrite rules to produce definitional equalities whenever applicable, enabling effective use of the least upper bound operator without the need to apply additional lemmas. The Kit least upper bound laws follow by case distinction on their Modes, but examining a proof in detail reveals a subtle issue. As an example, consider the proof for the ⊔-idem law. The remaining laws follow analogously.

```
⊔-idem {V^M} {{K}} = unique–V K
⊔-idem {T^M} {{K}} = unique–T K
```

If we split on the Mode M of the Kit K, we must prove in both cases that K is either V or T, according to the definition of _⊔_ and the chosen Mode. We run into the problem,

that K is inferred by instance resolution, and Agda cannot deduce that there is only one possible choice in both cases, V for V^M and T for T^M. Nor can the uniqueness of instances be assumed in general, as external code might declare additional instances.

Agda does not provide a notion of closed type classes, where instance declarations outside a module can be prohibited. In Haskell, such behavior can be encoded using closed type families [13]. In Agda, closed type classes can be simulated, by requiring a field K-lock: Lock of type private data Lock : Set where unlock : Lock in the construction of a record instance. Because Lock is private, it can only be constructed from within the library. Any attempt to declare an instance outside the library would fail, because the required field K-lock of type Lock cannot be filled.

We can now postulate the uniqueness of Kits after manually verifying that only two instances exist and that we rely solely on Kits provided by instance resolution, never creating them locally:

```
private postulate
  unique–V : (K : Kit V^M) → V ≡ K
  unique–T : (K : Kit T^M) → T ≡ K
```

The postulates are declared private and cannot cause inconsistencies for users outside the library. There also exists a version unique–K that computes whether a given Kit is V or T, based on its Mode.

4.4 Computing Compose Kits

Our original goal was to compute the third parameter of a Compose Kit. To motivate this and, more generally, the computation of Compose Kits from Kits, we consider a first attempt of the definition for the associativity law of the σ-calculus, which in our Kit-based setting corresponds to the associativity of Map composition:

```
associativity :
  {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{K₄ : Kit M₄}} {{K₅ : Kit M₅}} {{K₆ : Kit M₆}}
  {{C₁ : ComposeKit K₁ K₂ K₃}} {{C₂ : ComposeKit K₃ K₄ K₅}} {{C₃ : ComposeKit K₁ K₆ K₅}} {{C₄ : ComposeKit K₂ K
  (φ₁ : S₁ –[ K₁ ]→ S₂) (φ₂ : S₂ –[ K₂ ]→ S₃) (φ₃ : S₃ –[ K₄ ]→ S₄) →
  (φ₁ ;[ C₁ ] φ₂) ;[ C₂ ] φ₃ ≡ φ₁ ;[ C₃ ] (φ₂ ;[ C₄ ] φ₃)
```

Here C_1 to C_4 denote the four Compose Kits that are required for the compositions. For clarity we apply them to the composition operators explicitly, although instance resolution would perform this step automatically. This yields the most general formulation of the associativity law for Maps, where arbitrary Kits are admitted as long as both sides reduce to a K_5-Map. Our current formulation of the law, however, is not a valid rewrite rule. Both C_3 and C_4, required for the right side, appear only there, violating the condition that all arguments of a rewrite rule must occur on the left. Moreover, the law requires the instance argument K_6, used as the result of composing the K_2-Map with that of K_4, which is only used on the right. Because K_6 can be computed via the least upper bound operation we defined before, the main difficulty is to compute C_3 and C_4.

We can try to define a function $_;^K_$: (K_1 : Kit M_1) (K_2 : Kit M_2) → ComposeKit K_1 K_2 ($K_1 \sqcup K_2$), that evaluates to C^R, if K_1 has Mode V^M and to C^S, if K_1 has Mode T^M. The function can be readily defined, but its correctness relies not only on the uniqueness of Kit instances, but also on the uniqueness of Compose Kit instances:

```
private postulate
  unique–C^R : {{K : Kit M}} (C : ComposeKit V K K) → C ≡ C^R
  unique–C^S : {{K : Kit M}} {{C′ : ComposeKit K V K}} → (C : ComposeKit T K T) → C ≡ C^S
```

As before we require a value of Lock in the creation of Compose Kits. By manual inspection we confirm that C^R and C^S are the only instances and that none are created locally.

Yet the function $_;^K_$ is insufficiently general. Computing C_4 from the associativity law yields $C_4 = K_2 ;^K K_4 = \mathsf{ComposeKit}\ K_2\ K_4\ (K_2 \sqcup K_4)$. Consequently, C_3 evaluates to $C_3 = K_1 ;^K (K_2 \sqcup K_4) = \mathsf{ComposeKit}\ K_1\ (K_2 \sqcup K_4)\ (K_1 \sqcup (K_2 \sqcup K_4))$. The final Map on the right of the equation computes to a $(K_1 \sqcup (K_2 \sqcup K_4))$-Map, whereas the Map on the left is a K_5-Map. Adjusting the K_5 parameter in C_2 to $(K_1 \sqcup (K_2 \sqcup K_4))$ is not possible, as Compose Kits whose third parameter involves least upper bounds cannot be inferred by instance resolution. Defining instances that would permit the inference of Compose Kits with arbitrary least upper bound computation in the third parameter is hard and would increase instance resolution runtime dramatically. Additionally, Agda generally refused to infer such instances during higher-order pattern unification, performed when checking if a rewrite rule applies. A second approach, defining the least upper bound operation syntactically as a constructor in the Mode type, fails because the lattice laws can no longer be stated and rewritten. As a solution, we require an even more general, but also unsafe, function:

```
opaque private
  {-# NON_COVERING #-}
  _,_,_ : (K₁ : Kit M₁) (K₂ : Kit M₂) (K₃ : Kit M₃) → ComposeKit K₁ K₂ K₃
  _,_,_ {V^M} {V^M} {V^M} K₁ K₂ K₃ rewrite unique–K K₁ | unique–K K₂ | unique–K K₃ = C^R
  _,_,_ {V^M} {T^M} {T^M} K₁ K₂ K₃ rewrite unique–K K₁ | unique–K K₂ | unique–K K₃ = C^R
  _,_,_ {T^M} {V^M} {T^M} K₁ K₂ K₃ rewrite unique–K K₁ | unique–K K₂ | unique–K K₃ = C^S
  _,_,_ {T^M} {T^M} {T^M} K₁ K₂ K₃ rewrite unique–K K₁ | unique–K K₂ | unique–K K₃ = C^S
```

Because it is declared non-covering, evaluation may get stuck, yielding a value for a Compose Kit that should not exist. For this reason, every use of the function requires manual verification. Computed Compose Kits are reduced to the actual instances, when applicable, using the rewrite rules $\{\{K : \mathsf{Kit}\ M\}\} \to \mathsf{V}\ ,\ K\ ,\ K \equiv \mathsf{C}^R\ \{\{K = K\}\}$ for C^R and $\{\{K : \mathsf{Kit}\ M\}\} \to \mathsf{T}\ ,\ K\ ,\ \mathsf{T} \equiv \mathsf{C}^S\ \{\{K = K\}\}\ \{\{C = K\ ,\ \mathsf{V}\ ,\ K\}\}$ for C^S. The unsafe function is only available from within the library, but we can define the safe version from before as a special case of the unsafe function: $K_1 ;^K K_2 = K_1\ ,\ K_2\ ,\ (K_1 \sqcup K_2)$. We can now state a version of the associativity law eligible for rewriting:

```
associativity :
  {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}} {{K₄ : Kit M₄}} {{K₅ : Kit M₅}}
  {{C₁ : ComposeKit K₁ K₂ K₃}} {{C₂ : ComposeKit K₃ K₄ K₅}}
  (φ₁ : S₁ –[ K₁ ]→ S₂) (φ₂ : S₂ –[ K₂ ]→ S₃) (φ₃ : S₃ –[ K₄ ]→ S₄) →
  -- SAFETY: Assume the incoming Compose Kits C₁ and C₂ are valid.
  --         By case analysis on K₁, K₂, K₃, K₄ and K₅, the definition of _⊔_ and
  --         the uniqueness of (Compose-) Kits, we know the outgoing Compose Kits
  --         K₁ , (K₂ ⊔ K₄) , K₅ and K₂ , K₄ , (K₂ ⊔ K₄) are valid too. □
  (φ₁ ; φ₂) ; φ₃ ≡ φ₁ ;[ K₁ , (K₂ ⊔ K₄) , K₅ ] (φ₂ ;[ K₂ , K₄ , (K₂ ⊔ K₄) ] φ₃)
```

With the unsafe operator, we can connect $K_1 \sqcup (K_2 \sqcup K_4)$ with K_5 in the outer Compose Kit on the right side of the equation. It remains to check that whenever the incoming Compose Kits C_1 and C_2 are valid, the outgoing ones, computed using our unsafe function, are as

well. By exhaustively checking all eight valid combinations of values for C_1 and C_2, we confirm that in each case the outgoing Compose Kits are valid. Inside the associativity proof, the remaining cases must be refuted under the assumption of valid and unique incoming Compose Kits, and the non-existence of invalid Compose Kits:

```
private postulate
  impossible–C^SKR : {{K : Kit M}} → ¬ ComposeKit T K V
  impossible–C^KSR : {{K : Kit M}} → ¬ ComposeKit K T V
  impossible–C^RRS : ¬ ComposeKit V V T
```

As before, each use of these postulates requires manual verification to avoid inconsistencies, but cannot leak to the user, because they are declared private.

All other σ-calculus laws, generalized to Maps, follow the same structure as the associativity law. On the left side of the equations we rely on Kits and Compose Kits inferred via instance resolution, while on the right we compute the resulting Kits and Compose Kits using the least upper bound operation together with the unsafe _,_,_ operator, where we manually verify that its use is sound. The proofs are carried out by case analysis on the Modes of the Kit instance arguments and rely on the uniqueness of (Compose-) Kits as well as the refutation of invalid ComposeKit Kits. Apart from these aspects, their structure closely resembles the familiar separated proofs for renamings and substitutions.

4.5 The Scoped and Multi-Sorted σ-Calculus with Kits and Maps

Figure 5 states the σ-calculus laws in the Kit-setting. The figure lists the symbols and laws as they are used in AGDASUBST. For readability we omit the Kit and ComposeKit instance arguments. In the library, Kits and ComposeKits carry data, and Kits must be disambiguated on the type level by their Modes. Here we treat Kits as markers, which lets us elide the Mode lattice laws. Similarly, ComposeKits serve as markers for admissible compositions and are written explicitly for each operator in square brackets.

The symbols are grouped into several categories. Sorts and scopes provide intrinsic scoping at the type level. Variables are supplied by the library, while terms are user-defined, subject only to the requirement of a variable constructor of the appropriate form. We define the type of Kits together with the Variable Kit, the Term Kit, and the least upper bound operation. We then define the type of Maps and the four Map primitives, parameterized by Kits. The operator wk is specialized directly to a renaming to avoid duplication, as it can always be embedded as a substitution by composition with the identity substitution. When the Kit in the type index of Maps is known to be V or T, we use the variables ρ and σ respectively. From Kits we also obtain the combined type of variables and terms, together with the unified lookup/traversal operator. While there are distinct type definitions for variables, terms, and variables-or-terms, there is only a single type definition for Maps, specialized through the Kit index. There is no technical obstacle to unifying variables and terms in the same way. However, unlike renamings, substitutions, and Maps, terms are user-defined, whereas variables are provided by the library and therefore justify a distinct type definition.

We already discussed the **Kit Laws** in Section 4.3, so we now turn to the remaining ones. We begin with the type-level reduction laws img^V and img^T inside the **Definitional Laws**, which rewrite the type of variables-or-terms to either the type of variables or the type of terms when the Kit index is known to be V or T, respectively.

Next comes the definition of lookup on the identity substitution id^S. It is given in terms of the unified lookup/traversal operator. The laws wk, ext_0, and ext_s are the direct

generalizations of their familiar counterparts for renaming and substitution.

Lookup on the identity renaming does not require a separate law, as it is already subsumed by the generalized right-id from the **Monad Laws**. The right-id law accounts for both a term traversed by an identity substitution and a variable looked up in the identity renaming. This coincidence suggests a deeper connection between definitional laws and monad laws when there is only a single unified lookup/traversal operator. We can confirm our suspicion when examining the comp and compositionality law. There exists a fundamental symmetry between the definition of composition, given via the unified lookup/traversal operator, and the compositionality law in full generality:

comp–general :
x/t &/⋯[K_3 , K_4 , K_5] (φ_1 ;[K_1 , K_2 , K_3] φ_2) = (x/t &/⋯[K_1 , K_2 , K_3] φ_1) &/⋯[K_3 , K_4 , K_5] φ_2

compositionality–general :
(x/t &/⋯[K_1 , K_2 , K_3] φ_1) &/⋯[K_3 , K_4 , K_5] φ_4 ≡ x/t &/⋯[K_1 , $K_2 \sqcup K_4$, K_5] (φ_1 ;[K_2 , K_4 , ($K_2 \sqcup K_4$)] φ_2)

Viewed side by side, these laws exhibit an inherent symmetry. If both were admitted into the rewrite system, they would trigger infinite loops. This symmetry is no accident, intuitively, one law pushes variables into maps, while the other pushes maps into other maps, away from terms. The unification of lookup and traversal into a single operator makes this symmetry explicit. To avoid non-termination, we adopt the following heuristic:

- In variable position ($K_5 = \mathsf{V}$) use the definition of composition. In this case all other indices $K_1, \ldots, K_4$ are forced to V, and x/t must be a variable. This yields exactly the specialized comp law shown in the figure.
- In term position ($K_5 = \mathsf{T}$) use compositionality. Here no restrictions on the other indices arise, so the law is preserved in most of its generality and appears under the Monad Laws.

If neither condition applies (K_5 not fixed), the user must explicitly choose from the fully general laws which law to apply, either by equational reasoning or by using rewrite.

With the position heuristic the comp law is rather restricted. We now miss cases where we would make progress by applying the comp law, when φ_1 and φ_2 are not both renamings. Hence, we consider all situations where progress would be made directly after applying comp, by specializing on φ_1. The laws, where φ_1 or φ_2 is fixed to a primitive and progress can be inlined, appear under the name **Specialization Laws**.

The **Interaction Laws** are the direct generalizations of their original σ-calculus counterparts to Kits. Next come the **Traversal Laws**. Because we are independent of the object syntax except for the required variable constructor, we only show the law specific to the traversal of a variable. When a variable constructor is applied to a map, the variable it holds is mapped accordingly. To turn the result back into a term, as it may otherwise be either a variable or a term, depending on the map, we subsequently apply the identity substitution.

To illustrate another case for traversal, consider an object language that supports λ-abstractions. The corresponding user-supplied traversal function, needed to construct the Term Kit, would then include a clause (λx e)⋯ φ = λx (e ⋯ (φ ↑ expr)). However, the clause does not directly corresponds to a traversal law. It must first be lifted from the traversal operator to the unified lookup/traversal operator: (λx e) &/⋯ φ ≡ λx (e &/⋯ (φ ↑ expr)). The equation can be proven by reflexivity, when we unfold the definition of the unified lookup/traversal operator. It can then be added to our equational theory.

Finally, the **Coincidence Laws** generalize from renamings to Maps and enable Maps to be pulled out of substitutions when possible. As discussed earlier, these laws do not exhaustively describe every possible way in which Maps can be extracted from substitutions.

4.6 Discussion

The Kit-based version of the σ-calculus laws required unsafe computation and a careful formulation of the laws so that they remain valid rewrite rules while still delegating Kit and ComposeKit inference to the instance resolution algorithm. We believe the Kit-based variant, derived from the σ-calculus used in AUTOSUBST, remains terminating and confluent, even though Agda's built-in checker cannot verify confluence. From the Kit-based laws, we can readily derive the specialized laws from Section 3 and rewrite them if we wish to reason about renamings and substitutions separately.

The key difficulty in adapting the laws to the Kit-setting emerges from the symmetry between the general definition of composition comp and the compositionality law. Our solution is a heuristic that resolves this conflict by restricting the laws to cases where the context is known to be variable or term position. When the position is unknown, the flow of the laws becomes ambiguous, and the user must resort to manual application.

Compared to the laws from Section 3, the Kit-based formulation has the advantage that properties can be established directly for Maps, without requiring separate but structurally identical proofs for renamings and substitutions.

5 Case Studies

We present two examples illustrating the usage of the AGDASUBST library. The first concerns proofs of subject reduction for System F with subtyping. We begin by a presentation of the proof using the laws from Section 3.2 and then revisit it using the Kit-based laws from Section 4.5. We then extend the latter version with subtyping. The second example, based on work by [3], provides an instantiation for a generic universe of scoped syntax.

5.1 System F

Recall the multi-sorted and scoped syntax from the right side of Figure 2. We outline the proof of subject reduction, first using the laws from Section 3.2 and then using the AGDASUBST library. The type system and semantics are the same in both cases and are presented once in Figure 6. In both settings, we use the functions weaken : $S \vdash s \to (s' :: S) \vdash s$ and _[_] : $(s' :: S) \vdash s \to S \vdash s' \to S \vdash s$ to increment all variable indices within a term and to substitute a term for the most recently bound variable in another term. These functions can be defined directly from the primitives in both cases. Within the typing relation, we rely on the lookup relation _∋_:_ : Ctx $S \to S \ni s \to S :\vdash s \to$ Set to retrieve the type of a variable from the typing context. The type of the lookup and typing relation is expressed using the _:⊢_ operator. This operator computes the sort of the third index of the relations based on the sort of the second. With the _:⊢_ operator, we can define a single lookup and typing relation for well-typed expressions and well-kinded types. For System F, it can be defined as follows:

```
↑ᵗ_ : Sort → Sort
↑ᵗ expr = type
↑ᵗ type = kind
↑ᵗ kind = kind

_:⊢_ : Scope → Sort → Set
S :⊢ s = S ⊢ (↑ᵗ s)
```

Separated Renamings and Substitutions

We have already seen the instantiation of the σ-calculus for System F, where renamings and substitutions are treated separately, in Section 3.2. To establish subject reduction, we first prove that renaming and substitution preserves typing. The proofs for the preservation lemmas can be found in Figure 7. We prove renaming preserves typing (_⊢⋯^{R}_), and then, using this result in the proof for the fact that lifting of a substitution preserves typing (⊢↑^{S}, omitted), show that general substitution preserves typing (_⊢⋯^{S}_) as well.

The types of both proofs rely on the types for well-typed renamings _:_→^{R}_ : S_1 →^{R} S_2 → Ctx S_1 → Ctx S_2 → Set and well-typed substitutions _:_→^{S}_ : S_1 →^{S} S_2 → Ctx S_1 → Ctx S_2 → Set, whose definitions are omitted here. In the cases for variables, we look up the well-typed variable in the corresponding renaming or substitution. When going under a binder, we apply the preservation lemma for lifting renamings and substitutions. Thanks to the automatic application of the σ-calculus laws, both proofs follow directly by induction, including the cases under binders.

Using the two results, we can then prove subject reduction for System F:

```
subject-reduction :
  Γ ⊢ e : t →
  e ↪ e′ →
  Γ ⊢ e′ : t
subject-reduction (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ v₂) = _⊢⋯ˢ_ {σ = e₂ ·ˢ idˢ} (⊢[] ⊢e₂) ⊢e₁
subject-reduction (⊢▪ {t = t} (⊢Λ ⊢e) ⊢t ⊢t') β-Λ       = _⊢⋯ˢ_ {σ = t ·ˢ idˢ} (⊢[] ⊢t) ⊢e
subject-reduction (⊢· ⊢e₁ ⊢e₂)   (ξ-·₁ e₁↪e)           = ⊢· (subject-reduction ⊢e₁ e₁↪e) ⊢e₂
subject-reduction (⊢· ⊢e₁ ⊢e₂)   (ξ-·₂ e₂↪e x)         = ⊢· ⊢e₁ (subject-reduction ⊢e₂ e₂↪e)
subject-reduction (⊢▪ ⊢e ⊢t ⊢t') (ξ-▪ e↪e')             = ⊢▪ (subject-reduction ⊢e e↪e') ⊢t ⊢t'
```

All cases proceed without any explicit application of substitution lemmas. In particular, in the β-reduction cases, we can directly apply the substitution-preserves-typing lemma, whereas normally the interaction law would need to be applied manually as well.

Using Agdasubst

We first instantiate the library by providing the traversal function and the syntax-dependent laws. For System F, the traversal function is given by:

```
_⋯_ : ∀ {{K : Kit M}} → S₁ ⊢ s → S₁ –[ K ]→ S₂ → S₂ ⊢ s
(` x)        ⋯ φ = `/id (x & φ)
(λx e)       ⋯ φ = λx (e ⋯ (φ ↑ _))
(e₁ · e₂)    ⋯ φ = (e₁ ⋯ φ) · (e₂ ⋯ φ)
(t₁ ⇒ t₂)    ⋯ φ = (t₁ ⋯ φ) ⇒ (t₂ ⋯ φ)
(Λα e)       ⋯ φ = Λα (e ⋯ (φ ↑ _))
(∀[α: k ] t) ⋯ φ = ∀[α: (k ⋯ φ) ] (t ⋯ (φ ↑ _))
(e ▪ t)      ⋯ φ = (e ⋯ φ) ▪ (t ⋯ φ)
⋆            ⋯ φ = ⋆
```

In the case of variables, we convert the result of the lookup on the Map, whether a variable or a term, into a term. We then proceed to prove the monad laws:

```
{-# REWRITE id↑≡id #-}
⋯-right-id :
  ∀ {{K : Kit M}}
    (t : S ⊢ s) →
    t ⋯ id ≡ t
```

```
{-# REWRITE dist-↑-; #-}
⋯-compositionality :
  ∀ {{K₁ : Kit M₁}} {{K₂ : Kit M₂}} {{K₃ : Kit M₃}}
    {{C : ComposeKit K₁ K₂ K₃}}
    (t : S₁ ⊢ s) (φ₁ : S₁ -[ K₁ ]→ S₂) (φ₂ : S₂ -[ K₂ ]→ S₃) →
    (t ⋯ φ₁) ⋯ φ₂ ≡ t ⋯ (φ₁ ; φ₂)
```

We omit the proofs, but they follow directly by structural induction because the library already provides and permits rewriting of the necessary laws. Specifically, for right identity under a binder, we rely on id↑≡id : (id/‘ (zero {$s = s$} {$S = S$})) · (id ;wk) ≡ id, and for compositionality under a binder, we rely on dist-↑-; : ((φ_1 ↑ s) ; (φ_2 ↑ s)) ≡ ((φ_1 ; φ_2) ↑ s). Both additional laws follow directly from the rules of the σ-calculus and do not cause any issues when admitted as rewrite rules, in addition to the σ-calculus laws, rewritten later. We omit the syntax-dependent laws needed to lift the traversal operator to the unified lookup/traversal operator, when traversing a term. To complete the instantiation, lookup and traversal are redefined using the unified lookup/traversal operator:

```
_&_ : {{K : Kit M}} → S₁ ∋ s → S₁ -[ K ]→ S₂ → S₂ ∋/⊢[ K ] s
_&_ = _&/⋯_
```

```
_⋯_ : {{K : Kit M}} → S₁ ⊢ s → S₁ -[ K ]→ S₂ → S₂ ⊢ s
_⋯_ {{K}} = _&/⋯[ T ;ᴷ K ]_
```

To prove that renaming and substitution preserve typing in a single lemma, we make use of Typing Kits, following the approach of [22]. Typing Kits abstract over typed renamings and substitutions depending on a Kit, in a way analogous to how Maps abstract over renamings and substitutions based on a Kit. Within the library, they are instantiated exactly twice, once for the Variable Kit and once for Term Kit. For a more detailed description, we refer the reader to the already mentioned work. The unified proof that Maps preserve typing mirrors the shape of the separate proofs:

```
_⊢⋯_ : ∀ {{K : Kit M}} {{TK : TypingKit K}}
  {Γ₁ : Ctx S₁} {Γ₂ : Ctx S₂}
  {e : S₁ ⊢ s} {t : S₁ :⊢ s} {φ : S₁ -[ K ]→ S₂} →
  Γ₁ ⊢ e : t →
  φ : Γ₁ -[ TK ]→ Γ₂ →
  Γ₂ ⊢ (e ⋯ φ) : (t ⋯ φ)
⊢‘ {x = x} ⊢x ⊢⋯ ⊢φ = ⊢‘/id (⊢φ x _ ⊢x)
⊢λ ⊢e       ⊢⋯ ⊢φ = ⊢λ (⊢e ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢Λ ⊢e       ⊢⋯ ⊢φ = ⊢Λ (⊢e ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢· ⊢e₁ ⊢e₂  ⊢⋯ ⊢φ = ⊢· (⊢e₁ ⊢⋯ ⊢φ) (⊢e₂ ⊢⋯ ⊢φ)
⊢▪ ⊢e ⊢t ⊢t′ ⊢⋯ ⊢φ = ⊢▪ (⊢e ⊢⋯ ⊢φ) (⊢t ⊢⋯ ⊢φ) (⊢t′ ⊢⋯ (⊢φ ∋↑/⊢↑ _))
⊢⋆           ⊢⋯ ⊢φ = ⊢⋆
```

In the variable case, we need to convert a typed variable or term into a typed term. Agdasubst automatically solves all equations occurring from lifting. Compared with the proofs presented by [22] in Section 5.5[3], there is no need to manually move the Map under a weakening for binders or under a singleton substitution in the case of type application.

The proof of subject reduction is exactly the same as the one presented before. The only difference is that, instead of using the substitution preserves typing lemma, we use the more

[3] Additionally available online: Link to proof on GitHub

general lemma that Maps preserve typing in the cases of β-reductions:

```
subject-reduction (⊢· {e₂ = e₂} (⊢λ ⊢e₁) ⊢e₂) (β-λ _) = _⊢⋯_ {φ = e₂ · id} ⊢e₁ (⊢[] ⊢e₂)
subject-reduction (⊢▪ {t = t} (⊢Λ ⊢e) ⊢t ⊢t′) β-Λ     = _⊢⋯_ {φ = t · id} ⊢e (⊢[] ⊢t)
```

Normally, we would additionally need to apply the interaction law manually in the case of β-λ[4].

5.2 Generic Syntax

The class of object languages supported by Agdasubst can be described by the definition of a generic syntax à la [3], adapted to the intrinsically scoped setting [22]:

All laws required for instantiation of the Agdasubst library can be proven once and for all over the generic syntax. Users can define their own syntax directly from these building blocks without implementing traversals or reproving syntax-specific laws.

While the generic approach is very convenient, the main drawback is that Agda sometimes unfolds the pattern definitions inside holes, leaking details of the encoding, which results in proof states that are hard to read.

6 Further Work

This section outlines possible directions for improving Agdasubst in the future.

In contrast to most of the other substitution libraries, users of Agdasubst still need to manually provide the definition for traversal and the syntax-dependent proofs (Monad and Coincidence Laws). [22] has shown that definitions and proofs for scoped syntax can be derived automatically using Agda's reflection mechanism. There is no reason why the same approach could not be applied for the definitions and proofs required to instantiate Agdasubst.

[26] introduced several interesting extensions to the σ-calculus that could also be implemented in Agdasubst. One extension is support for variadic bindings (unknown number of variables bound), useful for languages with pattern matching. Another extension is support for lists of terms, allowing an unknown number of terms in the syntax tree. This would, for example, enable support for languages with n-ary application. The latter can already be encoded in the current system by inlining the list data structure directly into the syntax definition and treating them as terms of a new sort.

In its current state, Agdasubst completely lacks a meta-theoretical analysis. While we conjecture it to be terminating and confluent, it would be interesting to formally establish these properties.

As noted, Agdasubst is not complete when reasoning simultaneously about renamings, substitutions and Maps. It would therefore be worthwhile to investigate additional laws, in the spirit of the coincidence laws, to increase the space of solvable equations. Autosubst 2 already provides a more sophisticated tactic for the separation of renamings from substitutions, which could serve as a useful starting point for deriving such additional laws in Agdasubst.

At the moment, the performance of Agdasubst is, to be honest, very poor. For instance, the proof of subject reduction for System F with subtyping requires several minutes to check. Our analysis shows that most of this time is spent on instance resolution and function coverage analysis (i.e., verifying that pattern matching clauses are exhaustive). Currently, we have no

[4] See the proof provided by [22]: Link to proof on GitHub

concrete strategy to improve these aspects, without decreasing user-experience, except that we know, that the addition of the coincidence laws dramatically increases type-checking time.

7 Related Work

Explicit Substitution Calculi

There exist a variety of explicit substitution calculi, but we focus on the original σ-calculus [1], which includes identity, extension, shifting, and composition primitives, and the σ_{SP}-calculus [23], which adds further reductions to ensure confluence. Only the σ_{SP}-calculus is both sound and complete, enabling all equations between terms to be derived via reduction. Both completeness and convergence are established through mechanized proofs [23, 26].

Abstractions over Variables/Terms and Renamings/Substitutions

We already discussed the original work on Kits [20] and their adaptation to multi-sorted and scoped syntax [22] throughout Section 4.

The intrinsically scoped version of this approach also appears in recent work, for example in a paper written by [5], where a uniform treatment of renaming and substitution is explored in an intrinsically typed setting. Their work also includes a structure and definitions similar to the structural kit order we defined in Section 4.3. They additionally unify variables and terms, which we deliberately do not do, as variables are library-defined while terms are user-supplied. A further application of the Kit approach is given by [31], who use it to reason about metatheory in the context of linear languages.

Another line of research concerns generic programming approaches [3, 4], which allow the construction of syntax trees from general building blocks supporting sums, products, and binder introduction. The generic programming approach also allows for a generic structure that abstracts not only over renamings and substitutions, but also over semantics and other relevant language structures. The generic syntax approach can be adapted to a scoped and multi-sorted setting [22]. Importantly, generic programming and the Agdasubst library are orthogonal, as shown in Section 5.2.

Substitution Libraries for Proof Assistants

We have already discussed Autosubst 2 [27, 24, 26] which forms the direct theoretical foundation of our work.

There exists another Agda library with a similar goal to Agdasubst, called Abstract Binding Trees, used for instance by [25]. It permits constructing an object language from a generic syntax, similar to the approach in Section 5.2, but it supports only extrinsic scoping and single-sorted syntaxes. The library provides abstractions over arbitrary predicates about the object language, rather than focusing on specific relations such as typing [22]. The library also offers incomplete experimental support for rewriting using σ-calculus laws, but only handles renamings and substitutions separately, similar to the approach in Section 3.2, rather than in a uniform treatment.

More distantly related is Nominal 2 [29], an Isabelle package that addresses the α-equivalence problem not via de Bruijn indices, but using nominal datatypes and freshness conditions. Their approach is closer to the intuitive notion of variables, but the tool does not provide any strategy for solving equations involving terms and substitutions.

Additionally, there are tools such as Needle and Knot [17], which is a code generator for unscoped syntax using de Bruijn indices. The tool generates substitution and interaction

lemmas for single-pointed substitutions for languages that feature multiple variable sorts and variadic binders.

8 Conclusion

We have shown how to take advantage of the σ-calculus laws to automatically solve equations inside proofs about syntaxes with binding in Agda, similar to AUTOSUBST 2 in Rocq.

We first embedded the σ-calculus with first-class renamings directly into Agda as reduction rules, using its rewrite mechanism to yield definitional equalities for a formalization of System F.

We then wanted to unify proofs about renamings and substitutions, which led to the current state of the AGDASUBST library. To achieve our goal, we introduced the notions of Kits and Maps from prior work and adapted the σ-calculus laws to a Kit and Map setting.

We demonstrated how AGDASUBST can be used to prove properties involving substitutions for formal languages, including subject reduction for System F with subtyping using a single substitution-preserves-typing lemma, and provided an instantiation of a scoped universe of syntaxes with binding.

The AGDASUBST library does not constitute a stable release candidate yet. We have outlined potential directions for future development and plan to continue improving the library over time.

References

1. M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, page 31–46, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/96709.96712.
2. Robin Adams. Formalized metatheory with terms represented by an indexed family of types. In Jean-Christophe Filliâtre, Christine Paulin-Mohring, and Benjamin Werner, editors, *Types for Proofs and Programs*, pages 1--16, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. URL: https://link.springer.com/chapter/10.1007/11617990_1.
3. Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type and scope safe universe of syntaxes with binding: Their semantics and proofs, 2021. URL: https://arxiv.org/abs/2001.11001, arXiv:2001.11001.
4. Guillaume Allais, James Chapman, Conor McBride, and James McKinna. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, CPP 2017, page 195–207, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3018610.3018613.
5. Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. Substitution without copy and paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)*, Birmingham, UK, 2025.
6. Henk Barendregt and Kees Hemerik. Types in lambda calculi and programming languages. In Neil Jones, editor, *ESOP '90*, pages 1--35, Berlin, Heidelberg, 1990. Springer Berlin Heidelberg. URL: https://repository.ubn.ru.nl/bitstream/handle/2066/17284/17284.pdf.
7. Stefano Berardi. Towards a mathematical analysis of the coquand-huet calculus of constructions and the other systems in barendregt's cube. *Technica1 report, Carnegie-Me11on University (USA) and Universita di Torino (Ita1y)*, 1988. URL: https://link.springer.com/chapter/10.1007/3-540-58140-5_33.
8. L. Cardelli, S. Martini, J.C. Mitchell, and A. Scedrov. An extension of system f with subtyping. *Information and Computation*, 109(1):4--56, 1994. URL: https://www.sciencedirect.com/science/article/pii/S0890540184710133, doi:10.1006/inco.1994.1013.

9 Jesper Cockx. Type Theory Unchained: Extending Agda with Userined Rewrite Rules. In Marc Bezem and Assia Mahboubi, editors, *25th International Conference on Types for Proofs and Programs (TYPES 2019)*, volume 175 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1--2:27, Dagstuhl, Germany, 2020. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2019.2, doi:10.4230/LIPIcs.TYPES.2019.2.

10 Jesper Cockx and Andreas Abel. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi:10.1145/3236770.

11 Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi:10.1145/3434341.

12 Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM*, 43(2):362–397, March 1996. doi:10.1145/226643.226675.

13 Richard A. Eisenberg, Dimitrios Vytiniotis, Simon Peyton Jones, and Stephanie Weirich. Closed type families with overlapping equations. *SIGPLAN Not.*, 49(1):671–683, January 2014. doi:10.1145/2578855.2535856.

14 Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Phd thesis, Éditeur inconnu, 1972. URL: https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf.

15 Healfdene Goguen and James McKinna. Candidates for substitution. *LFCS report series-Laboratory for Foundations of Computer Science ECS LFCS*, 1997. URL: https://www.lfcs.inf.ed.ac.uk/reports/97/ECS-LFCS-97-358/.

16 Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420*, 2022. URL: https://arxiv.org/abs/2210.05420.

17 Steven Keuchel, Stephanie Weirich, and Tom Schrijvers. Needle & knot: Binder boilerplate tied up. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*, page 419–445, Berlin, Heidelberg, 2016. Springer-Verlag. doi:10.1007/978-3-662-49498-1_17.

18 D. E. Knuth and P. B. Bendix. *Simple Word Problems in Universal Algebras*, pages 342--376. Springer Berlin Heidelberg, Berlin, Heidelberg, 1983. doi:10.1007/978-3-642-81955-1_23.

19 Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73--118. Elsevier, 1975. URL: https://www.sciencedirect.com/science/article/pii/S0049237X08719451, doi:10.1016/S0049-237X(08)71945-1.

20 Conor McBride. Type-preserving renaming and substitution. Unpublished manuscript, 2005. URL: http://strictlypositive.org/ren-sub.pdf.

21 John C. Reynolds. Towards a theory of type structure. In B. Robinet, editor, *Programming Symposium*, pages 408--425, Berlin, Heidelberg, 1974. Springer Berlin Heidelberg. URL: https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf.

22 Hannes Saffrich. Abstractions for Multi-Sorted Substitutions. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 32:1--32:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.32, doi:10.4230/LIPIcs.ITP.2024.32.

23 Steven Schäfer, Gert Smolka, and Tobias Tebbi. Completeness and decidability of de bruijn substitution algebra in coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs*, CPP '15, page 67–73, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2676724.2693163.

24 Steven Schäfer, Tobias Tebbi, and Gert Smolka. Autosubst: Reasoning with de bruijn terms and parallel substitutions. In Christian Urban and Xingyuan Zhang, editors, *Interactive Theorem Proving*, pages 359--374, Cham, 2015. Springer International Publishing. URL: https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf.

25 Jeremy G. Siek and Tianyu Chen. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming*, 31:e30, 2021. doi:10.1017/S0956796821000241.

26 Kathrin Stark. *Mechanising Syntax with Binders in Coq*. PhD thesis, Saarland University, Saarbrücken, Germany, January 2020. Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.). URL: https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf.

27 Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: reasoning with multi-sorted de bruijn terms and vector substitutions. CPP 2019, page 166–180, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3293880.3294101.

28 Jan Terlouw. Een nadere bewijstheoretische analyse van gstt's. *Manuscript (in Dutch)*, 1989. URL: https://pure.tue.nl/ws/files/2226689/9613618.pdf.

29 Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. Nominal 2. *Archive of Formal Proofs*, February 2013. https://isa-afp.org/entries/Nominal2.html, Formal proof development.

30 P. Wadler and S. Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/75277.75283.

31 James Wood and Robert Atkey. A linear algebra approach to linear metatheory. *Electronic Proceedings in Theoretical Computer Science*, 353:195–212, December 2021. URL: http://dx.doi.org/10.4204/EPTCS.353.10, doi:10.4204/eptcs.353.10.

```
opaque
  _→ᴿ_ : Scope → Scope → Set
  S₁ →ᴿ S₂ = ∀ {s} → S₁ ∋ s → S₂ ∋ s

  _&ᴿ_ : S₁ ∋ s → (S₁ →ᴿ S₂) → S₂ ∋ s
  x &ᴿ ρ = ρ x

  idᴿ : S →ᴿ S
  idᴿ x = x

  wk : S →ᴿ (s :: S)
  wk x = suc x

  _·ᴿ_ : S₂ ∋ s → (S₁ →ᴿ S₂) → ((s :: S₁) →ᴿ S₂)
  (x ·ᴿ _) zero    = x
  (_ ·ᴿ ρ) (suc x) = ρ x

  _;ᴿᴿ_ : (S₁ →ᴿ S₂) → (S₂ →ᴿ S₃) → (S₁ →ᴿ S₃)
  (ρ₁ ;ᴿᴿ ρ₂) x = ρ₂ (ρ₁ x)
-- opaque end

_↑ᴿ_ : (S₁ →ᴿ S₂) → ∀ s → ((s :: S₁) →ᴿ (s :: S₂))
ρ ↑ᴿ _ = zero ·ᴿ (ρ ;ᴿᴿ wk)

_···ᴿ_ : S₁ ⊢ s → (S₁ →ᴿ S₂) → S₂ ⊢ s
-- Traversal Laws
(` x)          ···ᴿ ρ = ` (x &ᴿ ρ)
(λx e)         ···ᴿ ρ = λx (e ···ᴿ (ρ ↑ᴿ _))
(Λα e)         ···ᴿ ρ = Λα (e ···ᴿ (ρ ↑ᴿ _))
(∀[α: k ] t)   ···ᴿ ρ = ∀[α: k ···ᴿ ρ ] (t ···ᴿ (ρ ↑ᴿ _))
(e₁ · e₂)      ···ᴿ ρ = (e₁ ···ᴿ ρ) · (e₂ ···ᴿ ρ)
(e ▪ t)        ···ᴿ ρ = (e ···ᴿ ρ) ▪ (t ···ᴿ ρ)
(t₁ ⇒ t₂)      ···ᴿ ρ = (t₁ ···ᴿ ρ) ⇒ (t₂ ···ᴿ ρ)
⋆              ···ᴿ ρ = ⋆
```

```
opaque
  unfolding _→ᴿ_ wk

  _→ˢ_ : Scope → Scope → Set
  S₁ →ˢ S₂ = ∀ {s} → S₁ ∋ s → S₂ ⊢ s

  _&ˢ_ : S₁ ∋ s → (S₁ →ˢ S₂) → S₂ ⊢ s
  x &ˢ σ = σ x

  idˢ : S →ˢ S
  idˢ = `_

  _·ˢ_ : S₂ ⊢ s → (S₁ →ˢ S₂) → ((s :: S₁) →ˢ S₂)
  (t ·ˢ _) zero    = t
  (_ ·ˢ σ) (suc x) = σ x

  _;ᴿˢ_ : (S₁ →ᴿ S₂) → (S₂ →ˢ S₃) → (S₁ →ˢ S₃)
  (ρ₁ ;ᴿˢ σ₂) x = σ₂ (ρ₁ x)

  _;ˢᴿ_ : (S₁ →ˢ S₂) → (S₂ →ᴿ S₃) → (S₁ →ˢ S₃)
  (σ₁ ;ˢᴿ ρ₂) x = (σ₁ x) ···ᴿ ρ₂
-- opaque end

_↑ˢ_ : (S₁ →ˢ S₂) → ∀ s → ((s :: S₁) →ˢ (s :: S₂))
(σ ↑ˢ _) = (` zero) ·ˢ (σ ;ˢᴿ wk)

_···ˢ_ : S₁ ⊢ s → (S₁ →ˢ S₂) → S₂ ⊢ s
-- Traversal Laws
(` x)          ···ˢ σ = (x &ˢ σ)
(λx e)         ···ˢ σ = λx (e ···ˢ (σ ↑ˢ _))
(Λα e)         ···ˢ σ = Λα (e ···ˢ (σ ↑ˢ _))
(∀[α: k ] t)   ···ˢ σ = ∀[α: k ···ˢ σ ] (t ···ˢ (σ ↑ˢ _))
(e₁ · e₂)      ···ˢ σ = (e₁ ···ˢ σ) · (e₂ ···ˢ σ)
(e ▪ t)        ···ˢ σ = (e ···ˢ σ) ▪ (t ···ˢ σ)
(t₁ ⇒ t₂)      ···ˢ σ = (t₁ ···ˢ σ) ⇒ (t₂ ···ˢ σ)
⋆              ···ˢ σ = ⋆

opaque
  unfolding _→ˢ_

  _;ˢˢ_ : (S₁ →ˢ S₂) → (S₂ →ˢ S₃) → (S₁ →ˢ S₃)
  (σ₁ ;ˢˢ σ₂) x = (σ₁ x) ···ˢ σ₂
```

Figure 3 Scoped and Multi-Sorted Renamings and Substitutions

```
-- Definitional Laws
id^R : x &^R id^R              ≡ x
wk : x &^R wk {s = s'} ≡ suc x
ext_0^R : zero &^R (x ·^R ρ)     ≡ x
ext_s^R : (suc x') &^R (x ·^R ρ) ≡ x' &^R ρ

id^S : x &^S id^S              ≡ ‘ x
ext_0^S : zero &^S (t ·^S σ)     ≡ t
ext_s^S : (suc x) &^S (t ·^S σ) ≡ x &^S σ

comp^RR : x &^R (ρ1 ;^RR ρ2) ≡ (x &^R ρ1) &^R ρ2
comp^RS : x &^S (ρ1 ;^RS σ2) ≡ (x &^R ρ1) &^S σ2
comp^SR : x &^S (σ1 ;^SR ρ2) ≡ (x &^S σ1) ···^R ρ2
comp^SS : x &^S (σ1 ;^SS σ2) ≡ (x &^S σ1) ···^S σ2

-- Interaction Laws
comp-id_l^RR : id^R ;^RR ρ ≡ ρ; comp-id_l^RS : id^R ;^RS σ ≡ σ
comp-id_r^RR : ρ ;^RR id^R ≡ ρ
comp-id_r^SS : σ ;^SS id^S ≡ σ; comp-id_r^SR : σ ;^SR id^R ≡ σ
comp-id_l^SS : id^S ;^SS σ ≡ σ

associativity^RRR : (ρ1 ;^RR ρ2) ;^RR ρ3 ≡ ρ1 ;^RR (ρ2 ;^RR ρ3)
associativity^RRS : (ρ1 ;^RR ρ2) ;^RS σ3 ≡ ρ1 ;^RS (ρ2 ;^RS σ3)
associativity^RSR : (ρ1 ;^RS σ2) ;^SR ρ3 ≡ ρ1 ;^RS (σ2 ;^SR ρ3)
associativity^RSS : (ρ1 ;^RS σ2) ;^SS σ3 ≡ ρ1 ;^RS (σ2 ;^SS σ3)
associativity^SRR : (σ1 ;^SR ρ2) ;^SR ρ3 ≡ σ1 ;^SR (ρ2 ;^RR ρ3)
associativity^SRS : (σ1 ;^SR ρ2) ;^SS σ3 ≡ σ1 ;^SS (ρ2 ;^RS σ3)
associativity^SSR : (σ1 ;^SS σ2) ;^SR ρ3 ≡ σ1 ;^SS (σ2 ;^SR ρ3)
associativity^SSS : (σ1 ;^SS σ2) ;^SS σ3 ≡ σ1 ;^SS (σ2 ;^SS σ3)

distributivity^RR : (x ·^R ρ1) ;^RR ρ2    ≡ ((x &^R ρ2) ·^R (ρ1 ;^RR ρ2))
distributivity^RS : (x ·^R ρ1) ;^RS σ2    ≡ ((x &^S σ2) ·^S (ρ1 ;^RS σ2))
distributivity^SR : (t ·^S σ1) ;^SR ρ2    ≡ ((t ···^R ρ2) ·^S (σ1 ;^SR ρ2))
distributivity^SS : (t ·^S σ1) ;^SS σ2    ≡ ((t ···^S σ2) ·^S (σ1 ;^SS σ2))

interact^R : wk ;^RR (x ·^R ρ) ≡ ρ; interact^S : wk ;^RS (t ·^S σ) ≡ σ

η-id     : zero {S = S} {s = s} ·^R wk ≡ id^R
η-law^R : (zero &^R ρ) ·^R (wk ;^RR ρ)  ≡ ρ
η-law^S : (zero &^S σ) ·^S (wk ;^RS σ)  ≡ σ

-- Coincidence Laws
coincidence        : t ···^S (ρ ;^RS id^S) ≡ t ···^R ρ
coincidence-fold^V : t ···^S ((‘ x) ·^S (ρ ;^RS id^S)) ≡ t ···^R (x ·^R ρ)
coincidence-fold^T : t ···^S ((t' ···^R ρ) ·^S (ρ ;^RS id^S)) ≡ (t ···^S (t' ·^S id^S)) ···^R ρ

-- Monad Laws
right-id^R : t ···^R id^R ≡ t
right-id^S : t ···^S id^S ≡ t

compositionality^RR : (t ···^R ρ1) ···^R ρ2 ≡ t ···^R (ρ1 ;^RR ρ2)
compositionality^RS : (t ···^R ρ1) ···^S σ2 ≡ t ···^S (ρ1 ;^RS σ2)
compositionality^SR : (t ···^S σ1) ···^R ρ2 ≡ t ···^S (σ1 ;^SR ρ2)
compositionality^SS : (t ···^S σ1) ···^S σ2 ≡ t ···^S (σ1 ;^SS σ2)
```

```
{-# REWRITE
  id^R wk ext_0^R ext_s^R
  id^S ext_0^S ext_s^S
  comp^RR comp^RS comp^SR comp^SS

  comp-id_l^RR comp-id_l^RS comp-id_r^RR
  comp-id_r^SS comp-id_r^SR comp-id_l^SS
  associativity^RRR associativity^RRS
  associativity^RSR associativity^RSS
  associativity^SRR associativity^SRS
  associativity^SSR associativity^SSS
  distributivity^RR distributivity^RS
  distributivity^SR distributivity^SS
  interact^R interact^S
  η-id η-law^R η-law^S

  coincidence
  coincidence-fold^V coincidence-fold^T

  right-id^R right-id^S
  compositionality^RR compositionality^RS
  compositionality^SR compositionality^SS
#-}
```

Figure 4 The Scoped and Multi-Sorted σ-calculus with First-Class Renamings

Symbols

Sorts s
Sort : Set

.. *object language sorts:*

Scopes S
Scope : Set

[] : Scope
_::_ : Sort → Scope → Scope

Variables x
_∋_ : Scope → Sort → Set

zero : $(s :: S) \ni s$
suc : $S \ni s \to (s' :: S) \ni s$

Terms t
_⊢_ : Scope → Sort → Set

‘_ : $S \ni s \to S \vdash s$

.. *object language:*

Kits K
Kit : Set

V : Kit
T : Kit
_⊔_ : Kit → Kit → Kit

Maps φ
_–[_]→_: Scope → Kit → Scope → Set

id : S –[K]→ S
wk : S –[V]→ $(s :: S)$
_·_ : $S_2 \ni/\vdash[\ K\ ]\ s \to S_1$ –[K]→ $S_2 \to (s :: S_1)$ –[K]→ S_2
_;_ : S_1 –[K_1]→ $S_2 \to S_2$ –[K_2]→ $S_3 \to S_1$ –[K_3]→ S_3

Variables or Terms x/t
_∋/⊢[_]_ : Scope → Kit → Sort → Set

_&/⋯_ : $S_1 \ni/\vdash[\ K_1\ ]\ s \to S_1$ –[K_2]→ $S_2 \to S_2 \ni/\vdash[\ K_3\ ]\ s$

Reduction

Kit Laws

bot$_r$: $K \sqcup$ V $\equiv K$
bot$_l$: V $\sqcup K \equiv K$
top$_r$: $K \sqcup$ T $\equiv$ T
top$_l$: T $\sqcup K \equiv$ T
idem : $K \sqcup K \equiv K$
assoc : $(K_1 \sqcup K_2) \sqcup K_3 \equiv K_1 \sqcup (K_2 \sqcup K_3)$

Definitional Laws

img^V : $S \ni/\vdash[\ \mathsf{V}\ ]\ s \equiv S \ni s$
img^T : $S \ni/\vdash[\ \mathsf{T}\ ]\ s \equiv S \vdash s$

id^S : x &/⋯[V , T , T] id[T] ≡ ‘ x
wk : x &/⋯[V , V , V] wk ≡ suc x
ext$_0$: zero &/⋯[V , K , K] (x/t ·[K] φ) ≡ x/t
ext$_s$: suc x' &/⋯[V , K , K] (x/t ·[K] φ) ≡ x' &/⋯[V , K , K] φ
comp : x &/⋯[V , V , V] (ρ_1 ;[V , V , V] ρ_2) ≡ (x &/⋯[V , V , V] ρ_1) &/⋯[V , V , V] ρ_2

Specialization Laws

comp$_r$–id^S : x &/⋯[V , T , T] (ρ_1 ;[V , T , T] id[T]) ≡ ‘ (x &/⋯[V , V , V] ρ_1)
comp$_l$–id^S : x &/⋯[V , T , T] (id[T] ;[T , V , T] ρ_2) ≡ ‘ (x &/⋯[V , V , V] ρ_2)
comp$_l$–wk : x &/⋯[V , K , K] (wk ;[V , K , K] φ) ≡ suc x &/⋯[V , K , K] φ'
comp$_l$–ext$_0$: zero &/⋯[V , K_3 , K_3] ((x/t_1 ·[K_1] φ_1) ;[K_1 , K_2 , K_3] φ_2) ≡ x/t &/⋯[K_1 , K_2 , K_3] φ_2
comp$_l$–ext$_s$: suc x' &/⋯[V , K_3 , K_3] ((x/t_1 ·[K_1] φ_1) ;[K_1 , K_2 , K_3] φ_2) ≡

Interaction Laws

comp-id$_l$: id[K_1] ;[K_1 , K_2 , K_2] φ ≡ φ
comp-id$_r$: φ ;[K_1 , K_2 , K_1] id[K] ≡ φ
associativity : (φ_1 ;[K_1 , K_2 , K_3] φ_2) ;[K_3 , K_4 , K_5] φ_4 ≡ φ_1 ;[K_1 , ($K_2 \sqcup K_4$) , K_5] (φ_2 ;[K_2 , K_4 , ($K_2 \sqcup K_4$)] φ_4)
distributivity : (x/t_1 ·[K_1] φ_1) ;[K_1 , K_2 , K_3] φ_2 ≡ (x/t_1 &/⋯[K_1 , K_2 , K_3] φ_2) ·[K_3] (φ_1 ;[K_1 , K_2 , K_3] φ_2)
interact : wk ;[V , K , K] (x/t ·[K] φ) ≡ φ
η–id : zero ·[V] wk ≡ id[V]
η–law : (zero &/⋯[V , K , K] φ') ·[K] (wk ;[V , K , K] φ') ≡ φ'

Coincidence Laws

coincidence : t &/⋯[T , T , T] (φ ;[K , T , T] id[T]) ≡ t &/⋯[T , K , T] φ
coincidence–fold^K : t &/⋯[T , T , T] ((x/t &/⋯[K , T , T] id[T]) ·[T] (φ ;[K , T , T] id[T])) ≡ (t &/⋯[T , K , T] (x/t ·[K] φ))
coincidence–fold^T : t &/⋯[T , T , T] ((t' &/⋯[T , K , T] φ) ·[T] (φ ;[K , T , T] id[T])) ≡ (t &/⋯[T , T , T] (t' ·[T] (id[T]))) &/⋯[T , K , T] φ

Traversal Laws

var : (x) &/⋯[T , K , T] φ ≡ (x &/⋯[V , K , K] φ) &/⋯[K , T , T] id[T]

.. *object language traversal:*

Monad Laws

right-id : x/t_1 &/⋯[K_1 , K_2 , K_1] id[K_2] ≡ x/t_1
compositionality : (x/t_1 &/⋯[K_1 , K_2 , K_3] φ_1) &/⋯[K_3 , K_4 , T] φ_4 ≡ x/t_1 &/⋯[K_1 , $K_2 \sqcup K_4$, T] (φ_1 ;[K_2 , K_4 , ($K_2 \sqcup K_4$)] φ_2

```
data _⊢_:_ : Ctx S → S ⊢ s → S :⊢ s → Set where
  ⊢` : ∀ {x : S ∋ s} {t} →
    Γ ∋ x : t →
    Γ ⊢ (` x) : t
  ⊢λ :
    (t ::t Γ) ⊢ e : (weaken t′) →
    Γ ⊢ (λx e) : (t ⇒ t′)
  ⊢Λ :
    (k ::t Γ) ⊢ e : t →
    Γ ⊢ (Λα e) : (∀[α: k ] t)
  ⊢· :
    Γ ⊢ e1 : (t1 ⇒ t2) →
    Γ ⊢ e2 : t1 →
    Γ ⊢ (e1 · e2) : t2
  ⊢▪ :
    Γ ⊢ e : (∀[α: k ] t′) →
    Γ ⊢ t : k →
    (k ::t Γ) ⊢ t′ : k′ →
    Γ ⊢ (e ▪ t) : (t′ [ t ])
  ⊢⋆ : {t : S ⊢ type} →
    Γ ⊢ t : ⋆
```

```
data Val : S ⊢ expr → Set where
  vλ : Val (λx e)
  vΛ : Val (Λα e)

data _↪_ : S ⊢ expr → S ⊢ expr → Set where
  β-λ :
    Val e2 →
    ((λx e1) · e2) ↪ (e1 [ e2 ])
  β-Λ :
    ((Λα e) ▪ t) ↪ (e [ t ])
  ξ-·1 :
    e1 ↪ e →
    (e1 · e2) ↪ (e · e2)
  ξ-·2 :
    e2 ↪ e →
    Val e1 →
    (e1 · e2) ↪ (e1 · e)
  ξ-▪ :
    e ↪ e′ →
    (e ▪ t) ↪ (e′ ▪ t)
```

Figure 6 Typing and semantics for System F

```
_⊢…R_ : ∀ {e : S1 ⊢ s} {t : S1 :⊢ s} →
  ρ : Γ1 →R Γ2 →
  Γ1 ⊢ e : t →
  Γ2 ⊢ (e …R ρ) : (t …R ρ)
⊢ρ ⊢…R (⊢` ⊢x) =
  ⊢` (⊢ρ _ _ _ ⊢x)
⊢ρ ⊢…R (⊢λ ⊢e)        =
  ⊢λ ((⊢↑R ⊢ρ _) ⊢…R ⊢e)
⊢ρ ⊢…R (⊢Λ ⊢e)        =
  ⊢Λ ((⊢↑R ⊢ρ _) ⊢…R ⊢e)
⊢ρ ⊢…R (⊢· ⊢e1 ⊢e2) =
  ⊢· (⊢ρ ⊢…R ⊢e1) (⊢ρ ⊢…R ⊢e2)
⊢ρ ⊢…R (⊢▪ ⊢e ⊢t ⊢t′) =
  ⊢▪ (⊢ρ ⊢…R ⊢e) (⊢ρ ⊢…R ⊢t)((⊢↑R ⊢ρ _) ⊢…R ⊢t′)
⊢ρ ⊢…R ⊢⋆              =
  ⊢⋆
```

```
_⊢…S_ : ∀ {σ : S1 →S S2} {e : S1 ⊢ s} {t : S1 :⊢ s} →
  σ : Γ1 →S Γ2 →
  Γ1 ⊢ e : t →
  Γ2 ⊢ (e …S σ) : (t …S σ)
⊢σ ⊢…S (⊢` ⊢x) =
  ⊢σ _ _ _ ⊢x
_⊢…S_ {σ = σ} ⊢σ (⊢λ ⊢e)        =
  ⊢λ (_⊢…S_ {σ = σ ↑S _} (⊢↑S {σ = σ} ⊢σ _) ⊢e)
_⊢…S_ {σ = σ} ⊢σ (⊢Λ ⊢e)        =
  ⊢Λ (_⊢…S_ {σ = σ ↑S _} (⊢↑S {σ = σ} ⊢σ _) ⊢e)
_⊢…S_ {σ = σ} ⊢σ (⊢· ⊢e1 ⊢e2) =
  ⊢· (_⊢…S_ {σ = σ} ⊢σ ⊢e1) (_⊢…S_ {σ = σ} ⊢σ ⊢e2)
_⊢…S_ {σ = σ} ⊢σ (⊢▪ ⊢e ⊢t ⊢t′) =
  ⊢▪ (_⊢…S_ {σ = σ} ⊢σ ⊢e) (_⊢…S_ {σ = σ} ⊢σ ⊢t)
    (_⊢…S_ {σ = σ ↑S _} (⊢↑S {σ = σ} ⊢σ _) ⊢t′)
_⊢…S_ ⊢σ ⊢⋆ =
  ⊢⋆
```

Figure 7 Renaming and Substitution Preserve Typing